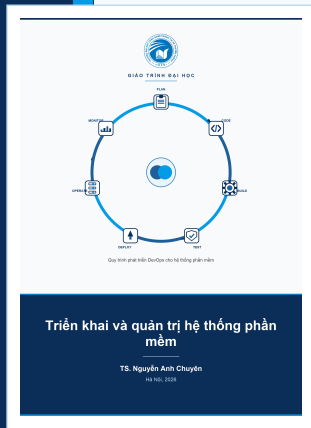




CHƯƠNG 9

Bảo mật & củng cố hệ thống

Triển khai và quản trị hệ thống phần mềm



Giới thiệu nội dung

Mục tiêu chương

- Giải thích mô hình CIA, tư duy Zero Trust và các nguy cơ phổ biến (sai cấu hình, chuỗi cung ứng, brute-force).
- Thiết lập tường lửa iptables theo nguyên tắc Default Drop và cấu hình SSH hardening.
- Triển khai và so sánh Fail2Ban với CrowdSec để chống brute-force, kiểm thử bằng Hydra.

Nội dung chính

- 1 Bối cảnh và nguy cơ an toàn hệ thống**
- 2 Kỹ thuật tấn công và phòng chống**
- 3 Tường lửa iptables trên Linux**
- 4 Củng cố bảo mật SSH**
- 5 Chống Brute-force: Fail2Ban và CrowdSec**

Vì sao bảo mật khép lại học phần?

- Các chương trước đã giúp triển khai, vận hành và giám sát hệ thống **chạy được và chạy tốt** trên máy chủ Linux.
- Một hệ thống chạy tốt nhưng bị chiếm quyền, mã hóa dữ liệu hay biến thành botnet vẫn là một hệ thống thất bại.
- Bảo mật không phải bước “thêm vào cuối” mà là điều kiện bắt buộc trước khi đưa hệ thống ra môi trường sản xuất.
- Chương này khép lại học phần bằng lớp phòng thủ cuối: nhận diện nguy cơ, kỹ thuật tấn công phổ biến và công cụ củng cố máy chủ Linux thực chiến.

Câu hỏi mở đầu

Nếu quên cấu hình tường lửa hay dùng mật khẩu SSH mặc định, hệ thống em vừa triển khai xong sẽ bị chiếm quyền trong bao lâu?

Ba trụ cột CIA và tư duy Zero Trust

- Ý nghĩa cốt lõi của bảo mật hệ thống nằm ở việc duy trì **bộ ba CIA**: tính bảo mật (Confidentiality), tính toàn vẹn (Integrity), tính sẵn sàng (Availability).
- Mô hình cũ “lâu đài và hào nước”: mặc định tin tưởng mọi thứ bên trong mạng nội bộ.
- Tư duy hiện đại: kiến trúc **Zero Trust** – “không bao giờ tin tưởng, luôn luôn xác thực”, áp dụng cho mọi yêu cầu dù đến từ trong hay ngoài mạng.
- Zero Trust giúp hạn chế **di chuyển ngang** của kẻ tấn công khi một mắt xích bị chiếm quyền.

Vì sao quan trọng

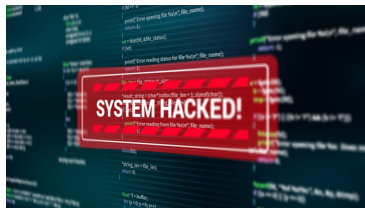
Làm việc từ xa và thiết bị cá nhân khiến ranh giới mạng doanh nghiệp không còn rõ ràng – Zero Trust thay thế cho ranh giới vật lý đã biến mất.

Bảo mật hệ thống: không chỉ là chống hacker

- **Hiệu suất & tài nguyên:** máy chủ không được củng cố dễ bị biến thành công cụ đào tiền điện tử (cryptojacking) hoặc mắt xích trong mạng botnet DDoS.
- Khi đó CPU, RAM, băng thông bị chiếm dụng, ảnh hưởng trực tiếp tới người dùng hợp pháp.
- **Khả năng chống chịu (resilience):** trước ransomware, bảo mật không chỉ là ngăn chặn mà còn là chuẩn bị cho kịch bản xấu nhất.
- Giải pháp sao lưu bất biến đảm bảo dữ liệu phục hồi được ngay cả khi kẻ tấn công chiếm quyền quản trị cao nhất.

Nguy cơ hàng đầu: sai cấu hình và chuỗi cung ứng

- **79%** các cuộc tấn công vào Linux khai thác dịch vụ cấu hình mặc định không an toàn, không dùng mã độc phức tạp.
- Tấn công **chuỗi cung ứng phần mềm**: nhắm vào thư viện mã nguồn mở, gói bên thứ ba thay vì tấn công trực tiếp mục tiêu.
- Case study: SolarWinds, Log4Shell – một sơ hở nhỏ gây tác động dây chuyền tới hàng chục nghìn tổ chức.



Nguy cơ mới: mã AI, vá lỗi chậm và con người

- **45%** mã nguồn do AI gợi ý chứa lỗ hổng nghiêm trọng (XSS, tiêm nhiễm mã lệnh) do dữ liệu huấn luyện chất lượng không đồng đều.
- “Ảo giác AI” gợi ý thư viện không tồn tại – kẻ tấn công tạo gói giả mạo trùng tên (typosquatting) để lừa cài đặt.
- Doanh nghiệp chậm áp dụng bản vá zero-day vì lo ngại tương thích và thời gian ngừng hoạt động – tạo ra “khoảng thời gian vàng” cho kẻ tấn công.
- Kỹ nghệ xã hội kết hợp **deepfake** biến con người thành mắt xích yếu nhất trong hệ thống phòng thủ.

Toàn cảnh các dạng tấn công khác cần lưu ý

Dạng tấn công	Đặc điểm chính
Ransomware tổng tiền ICS/OT	Vừa mã hóa dữ liệu vừa đe dọa rò rỉ thông tin nhạy cảm Hệ thống điều khiển công nghiệp kết nối IT nhưng khó vá lỗi, dễ bị rà quét
Đa vector	Kết hợp tấn công trong/ngoài, kỹ nghệ xã hội và lỗ hổng zero-day
AI/ML	Đầu độc dữ liệu huấn luyện, deepfake, tấn công vào hộp đen mô hình
Điện toán đám mây BEC	DDoS, tấn công API, Firewall Evasion, Man-in-the-Middle Lừa đảo tinh vi nhắm vào giao dịch tài chính với đối tác nước ngoài

SQL Injection: khai thác và phòng chống triệt để

- Kẻ tấn công lợi dụng việc **thiếu kiểm soát dữ liệu đầu vào** để chèn mã SQL độc hại vào truy vấn ứng dụng.
- Khi mã nguồn cộng chuỗi để tạo câu lệnh, chuỗi tấn công như `' OR 1=1 --` khiến điều kiện kiểm tra luôn đúng.
- Hệ quả: vượt qua xác thực hoặc trích xuất toàn bộ cơ sở dữ liệu.

Phòng chống triệt để

Dùng **Parameterized Queries** (Prepared Statements): tách biệt hoàn toàn phần lệnh và phần dữ liệu, ký tự đặc biệt của hacker không thể thay đổi cấu trúc câu lệnh gốc.

Ransomware thế hệ mới: tổng tiền kếp và BYOVD

- Không chỉ mã hóa dữ liệu mà còn đe dọa rò rỉ thông tin nhạy cảm nếu không trả tiền chuộc (“tổng tiền kếp”).
- Kỹ thuật **BYOVD** (Bring Your Own Vulnerable Driver): nạp driver có lỗ hổng để chiếm quyền mức kernel, vô hiệu hóa phần mềm bảo mật (EDR) trước khi mã hóa.
- Tỷ lệ sự cố có đánh cắp dữ liệu đã tăng từ 40% lên gần 80% trong những năm gần đây.

Phòng vệ tối ưu

Kết hợp **Defense-in-Depth** với **Immutable Backup**: sao lưu dạng WORM (Write Once, Read Many) – ngay cả khi hacker chiếm được quyền admin cao nhất cũng không thể xóa hay sửa bản sao lưu.

Brute-force SSH và tấn công dựa trên định danh

- Botnet tự động rà soát cổng 22 trên toàn dải IP công cộng, thử hàng triệu tổ hợp tên người dùng/mật khẩu.
- Phòng chống: vô hiệu hóa đăng nhập mật khẩu, chuyển sang khóa **Ed25519**, kết hợp **MFA** và công cụ cộng đồng như CrowdSec.
- Xu hướng mới: **tấn công dựa trên định danh** (identity-based) – hacker dùng thông tin đăng nhập đánh cắp để vào thẳng Cloud/SaaS, không để lại dấu vết trên máy trạm do tổ chức quản lý.
- Giải pháp: kiến trúc **XDR** giám sát hành vi bất thường xuyên suốt mạng, định danh và dịch vụ đám mây.

Case study: rò rỉ dữ liệu Snowflake (2024)

- Hơn 165 tổ chức lớn (AT&T, Ticketmaster...) trở thành nạn nhân của một chiến dịch tấn công nhắm vào nền tảng dữ liệu đám mây Snowflake.
- Kẻ tấn công **không khai thác lỗ hổng kỹ thuật** mà dùng tài khoản quản trị không bật MFA, với thông tin đăng nhập bị thu thập từ mã độc Infostealer trên máy cá nhân của nhân viên.
- Nguyên nhân gốc rễ: tin tưởng mù quáng vào mật khẩu truyền thống, thiếu giám sát định danh trên môi trường Cloud.

Bài học

Bảo mật máy chủ hiện đại không chỉ bảo vệ “phần mềm” mà phải tập trung vào quản trị **danh tính người dùng** và nguyên tắc đặc quyền tối thiểu.

Ba nguyên tắc nền tảng trước khi triển khai

- **Bảo mật là một quá trình liên tục** (process), không phải sản phẩm mua về là xong – cần giám sát và cập nhật không ngừng.
- Tư duy “**giả định bị xâm nhập**” (assume breach): luôn chuẩn bị lớp phòng thủ bên trong để cô lập thiệt hại khi lớp biên bị xuyên thủng.
- Sự kết hợp **con người – quy trình – công nghệ**: công nghệ mạnh đến đâu cũng thất bại nếu con người thiếu nhận thức và quy trình lỏng lẻo.

Từ nguyên lý đến thực hành

Phần tiếp theo áp dụng các nguyên tắc trên vào ba lớp phòng thủ cụ thể trên máy chủ Linux: tường lửa, SSH và chống brute-force.

Netfilter: framework lọc gói tin trong nhân Linux

- **Netfilter** là framework tích hợp sẵn trong Linux Kernel: lọc gói tin (packet filtering), NAT, theo dõi kết nối (connection tracking), chỉnh sửa header (packet mangling).
- Hoạt động qua các điểm đánh chặn (**hooks**) trên luồng xử lý mạng:

Hook	Vai trò
PREROUTING	Xử lý gói tin ngay khi nhận, trước khi định tuyến
INPUT	Gói tin gửi trực tiếp đến hệ thống cục bộ
FORWARD	Gói tin đi qua hệ thống (không dành cho máy hiện tại)
OUTPUT	Gói tin bắt nguồn từ chính hệ thống gửi đi
POSTROUTING	Xử lý sau định tuyến, trước khi đẩy ra card mạng

iptables: bảng, chuỗi, quy tắc

- **iptables** là công cụ userspace cấu hình Netfilter; **nftables** là công cụ kế nhiệm hiện đại, hiệu năng cao hơn, cú pháp hợp nhất.
- Cấu trúc: mỗi **bảng** (table) chứa các **chuỗi** (chains), mỗi chuỗi chứa các **quy tắc** (rules).
- Bảng **filter** có 3 chuỗi mặc định: **INPUT** (gói vào), **OUTPUT** (gói ra – ngăn máy chủ nhiễm mã độc trở thành trạm trung chuyển), **FORWARD** (gói đi xuyên qua, khi máy chủ là router/gateway).
- Mỗi quy tắc kiểm tra từ trên xuống; khớp thì thực thi **target**: **ACCEPT**, **DROP** (hủy âm thầm) hoặc **REJECT** (từ chối có phản hồi).

Triết lý thiết kế: Default Drop

- Sai lầm phổ biến: cấu hình tường lửa theo hướng “chặn cái xấu” (blacklisting).
- Tư duy hiện đại: nguyên tắc **Default Drop** (whitelisting) – mặc định từ chối mọi kết nối, chỉ mở đúng cổng dịch vụ cần thiết.
- Thu hẹp tối đa bề mặt tấn công: botnet rà quét tự động không thể tìm thấy dịch vụ nội bộ đang chạy ngầm.
- Ví dụ máy chủ web: chỉ mở 80 (HTTP), 443 (HTTPS), cổng SSH quản trị; khóa chặt MySQL (3306), Redis (6379) khỏi Internet.

Trước khi chạy DROP mặc định

Luôn đảm bảo đã cho phép kết nối SSH hiện tại trước khi áp policy DROP – tránh tự khóa mình khỏi hệ thống.

Thực hành: chính sách mặc định và loopback

Bước 1 – chính sách mặc định (giữ SSH hiện tại trước):

```
$sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
$sudo iptables -P INPUT DROP
$sudo iptables -P FORWARD DROP
$sudo iptables -P OUTPUT ACCEPT
```

Bước 2 – cho phép giao diện loopback (bắt buộc cho tiến trình nội bộ):

```
$sudo iptables -A INPUT -i lo -j ACCEPT
```

Thực hành: mở cổng dịch vụ và chống DDoS

Bước 3 – mở cổng SSH (whitelist IP) và Web:

```
$sudo iptables -A INPUT -p tcp -s 123.45.67.89 --dport 2849 -j ACCEPT
$sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
$sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
```

Bước 4 – giới hạn tốc độ chống SYN Flood:

```
$sudo iptables -A INPUT -p tcp --syn -m limit --limit 1/s --limit-burst 3
↪ -j ACCEPT
```

- Lưu quy tắc vĩnh viễn: cài `iptables-persistent`, chạy `sudo netfilter-persistent save`.
- Kiểm tra: `sudo iptables -L -n -v` (`-v` hiện lượng dữ liệu đã chặn).

Case study: củng cố máy chủ web thương mại điện tử

- Doanh nghiệp tại Việt Nam trên Ubuntu 24.04 bị rà quét chiếm quyền MySQL do để lộ cổng **3306** ra Internet.
- **Phân tích:** MySQL chỉ cần phục vụ ứng dụng chạy trên chính máy chủ đó (localhost).
- **Triển khai:** iptables chỉ mở cổng 443 và SSH tùy chỉnh; cổng 3306 chỉ **ACCEPT** từ giao diện **lo**.
- **Kết quả:** rà quét từ bên ngoài bị **DROP** hoàn toàn; điểm Hardening Index (đo bằng Lynis) tăng thêm 15 điểm.

SSH: mục tiêu hàng đầu và nguyên tắc “mất gốc”

- SSH luôn mở để nhận kết nối từ xa nên thường xuyên hứng chịu rà quét tự động từ botnet.
- Hơn 90% các cuộc tấn công SSH nhắm vào tài khoản **root** với mật khẩu yếu hoặc cấu hình mặc định.
- Nguyên tắc đầu tiên: **vô hiệu hóa đăng nhập root**, chuyển sang tài khoản thường có quyền `sudo` – buộc kẻ tấn công phải đoán thêm cả tên người dùng.
- Mật khẩu truyền thống đã lỗi thời trước brute-force hiện đại; xác thực khóa công khai là tiêu chuẩn vàng.

Ed25519, MFA và kỹ thuật ẩn mình

- Khuyến nghị dùng khóa **Ed25519** thay RSA cũ: nhỏ gọn, an toàn tương đương RSA 4096-bit, tốc độ xử lý vượt trội.
- Khóa SSH vẫn có rủi ro nếu máy quản trị viên nhiễm Infostealer làm lộ private key – cần thêm **MFA** (TOTP qua Google Authenticator...) cho hệ thống production.
- Đổi cổng mặc định (22 sang số ngẫu nhiên, VD 2849): loại bỏ 99% rà quét tự động hàng loạt, dù không chặn được hacker chuyên nghiệp có chủ đích.
- Whitelist IP truy cập SSH qua tường lửa/VPN: biện pháp kiểm soát biên mạnh nhất, khiến cổng SSH “vô hình” với phần còn lại của Internet.

Quản lý phiên và giám sát nhật ký SSH

- `MaxAuthTries`: giới hạn số lần thử đăng nhập sai; `MaxStartups`: giới hạn số kết nối đồng thời – làm chậm quá trình tấn công.
- Idle Timeout tự động ngắt phiên không hoạt động – tránh phiên bị bỏ quên trên máy công cộng.
- Ghi log mức `VERBOSE`: đầy đủ dấu vân tay khóa sử dụng và nguồn gốc kết nối, phục vụ điều tra sự cố.
- Kết hợp log tập trung và cảnh báo tự động (Telegram/Email) để phản ứng kịp thời.

Case study: chiếm quyền máy chủ production tại một startup

- Startup cấp quyền SSH **root** cho toàn đội kỹ thuật để thuận tiện bảo trì; một thành viên dùng mật khẩu yếu.
- Kẻ tấn công dùng công cụ **Hydra** brute-force thành công vào root; toàn bộ dữ liệu bị mã hóa bởi ransomware trong vài giờ.

Sai lầm

Bật root login, cho phép dùng mật khẩu, thiếu giám sát log tập trung.

Khắc phục

Vô hiệu hóa root login, chuyển hoàn toàn sang SSH key Ed25519 có passphrase, triển khai MFA cho toàn nhân viên – brute-force giảm **95%**.

Thực hành: tạo và triển khai khóa Ed25519

Bước 1 – tạo cặp khóa trên máy cá nhân (client):

```
$ssh-keygen -t ed25519 -o -a 100 -C "admin@yourdomain.com"
```

- `-o`: định dạng OpenSSH mới.
- `-a 100`: tăng số vòng KDF để chống brute-force khóa.
- `-C`: ghi chú định danh (email/thiết bị).

Đẩy khóa công khai lên máy chủ:

```
$ssh-copy-id -p 22 <username>@<server_ip>
```

Thực hành: cấu hình `sshd_config` an toàn

```
Port 2849
Protocol 2
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
PermitEmptyPasswords no
ClientAliveInterval 300
ClientAliveCountMax 0
MaxAuthTries 3
LogLevel VERBOSE
```

- Kiểm tra cú pháp trước khi restart: `sudo sshd -t`.
- Áp dụng: `sudo systemctl restart ssh`.

Tránh tự khóa mình

Giữ một phiên SSH đang mở cho đến khi đăng nhập thành công qua cổng mới ở cửa sổ khác.

Bốn nguyên tắc cần nhớ để bảo vệ SSH

- **Chìa khóa thay cho mật khẩu:** SSH key Ed25519 là rào cản vững chắc nhất chống lại brute-force.
- **Nguyên tắc “mất gốc”:** vô hiệu hóa root login buộc kẻ địch phải leo thang đặc quyền thêm bước, kéo dài thời gian phát hiện.
- **MFA là bắt buộc:** tấm khiên cuối cùng bảo vệ định danh trong kỷ nguyên mã độc đánh cắp khóa.
- **Ẩn mình thông minh:** đổi cổng và whitelist IP giúp giảm nhiễu hệ thống đáng kể.

Brute-force hiện đại: từ một IP sang hàng vạn IP

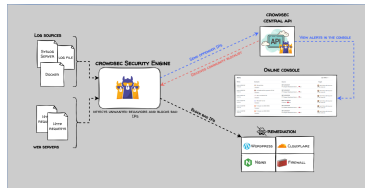
- Hơn 90% nỗ lực xâm nhập dịch vụ quản trị (SSH) nhắm vào root hoặc thông tin đăng nhập yếu, gây sụt giảm hiệu suất máy chủ do chiếm CPU/băng thông.
- Kẻ tấn công không còn dùng một IP thử hàng nghìn mật khẩu (dễ bị phát hiện) mà dùng **hàng vạn IP**, mỗi IP chỉ thử vài lần để né bộ lọc truyền thống.
- Buộc quản trị viên đổi tư duy: không chỉ phòng thủ cô lập mà phải dựa vào **threat intelligence** từ cộng đồng.

Fail2Ban: cơ chế lọc log truyền thống

- Rà soát liên tục log hệ thống (`/var/log/auth.log` trên Ubuntu, `/var/log/secure` trên CentOS).
- Khi số lần đăng nhập sai từ một nguồn vượt ngưỡng (`MaxRetry`) trong khoảng thời gian (`FindTime`), kích hoạt quy tắc tường lửa từ chối IP đó.
- **Hạn chế:** viết bằng Python + regex nên có thể nghẽn khi log lớn; mang tính **phòng thủ thụ động** – chỉ hành động sau khi đã hứng một lượng tấn công nhất định.

CrowdSec: threat intelligence và kiến trúc ba thành phần

- Viết bằng Go, xử lý JSON-native – nhanh hơn Fail2Ban tới **60 lần**, tốn ít tài nguyên hơn.
- **Acquisition:** thu log từ nhiều nguồn (SSH, Nginx, MySQL...).
- **Parsers & Scenarios:** nhận diện kịch bản tấn công (brute-force, SQLi...).
- **Bouncers:** lực lượng thực thi – chặn IP, yêu cầu CAPTCHA hoặc lọc qua WAF.



So sánh Fail2Ban và CrowdSec

Đặc điểm	Fail2Ban	CrowdSec
Ngôn ngữ	Python (chậm hơn)	Go (nhANH, hiệu năng cao)
Cơ chế	Cô lập, chỉ dựa trên máy chủ hiện tại	Cộng đồng, chia sẻ dữ liệu toàn cầu
Phạm vi	Chủ yếu Layer 3/4 (Firewall)	Đa tầng (CAPTCHA, WAF, Cloud)
Tài nguyên	Thấp, hợp VPS < 512MB RAM	Vừa phải, tối ưu hệ thống hiện đại

Thực hành: cấu hình Fail2Ban cho SSH

```
$sudo apt install fail2ban -y
$sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
$sudo nano /etc/fail2ban/jail.local
```

Trong `jail.local`, mục `[sshd]`:

```
[sshd]
enabled = true
port    = 2849
filter  = sshd
logpath = /var/log/auth.log
maxretry = 3
findtime = 600
bantime  = 3600
```

```
$sudo systemctl restart fail2ban
```

Thực hành: triển khai CrowdSec

```
$curl -s https://packagecloud.io/install/repositories/crowdsec/crowdsec/sc |  
↪ ript.deb.sh | sudo bash  
$sudo apt install crowdsec -y  
$sudo apt install crowdsec-firewall-bouncer-iptables -y
```

Kiểm tra trạng thái:

```
$sudo cscli decisions list  
$sudo cscli alerts list
```

Mở rộng

Với web server, nên cài thêm Nginx Bouncer để hiển thị CAPTCHA thay vì chặn hoàn toàn – tránh chặn nhầm người dùng hợp pháp.

Case study: chặn botnet 10.000 thiết bị

- Startup Việt Nam ghi nhận log SSH tăng vọt: hàng nghìn nỗ lực/phút từ IP trải khắp thế giới, mỗi IP chỉ thử 1 lần rồi biến mất.
- **Fail2Ban truyền thống bắt lỗi:** không đạt ngưỡng `maxretry` nên không kích hoạt chặn.
- **Giải pháp:** triển khai CrowdSec.
- **Kết quả:** chặn ngay **80%** IP tấn công nhờ Community Blocklist (dù chưa từng kết nối tới máy chủ trước đó); CPU giảm từ 90% xuống còn 5%.

Kiểm thử phòng thủ: Hydra và script mô phỏng botnet

Hydra (mô phỏng chuyên nghiệp):

```
$hydra -l admin -P passwords.txt \  
<server_ip> ssh -t 4
```

- **-l**: tên người dùng mục tiêu.
- **-P**: tệp danh sách mật khẩu.
- **-t 4**: giới hạn luồng song song.

Script Bash (mô phỏng rải rác):

```
for i in {1..5}; do  
  ssh -o BatchMode=yes \  
    -o ConnectTimeout=5 \  
    <user>@<server_ip> 2>&1 \  
    | grep "Permission denied"  
done
```

Vì sao cần cả hai

Hydra kiểm chứng ngưỡng **maxretry** của Fail2Ban; script rải rác mô phỏng hành vi phân tán mà CrowdSec được thiết kế để phát hiện.

Xác nhận kết quả kiểm thử

Với Fail2Ban:

```
$sudo fail2ban-client status sshd
```

- Kết quả kỳ vọng: IP máy trạm tấn công xuất hiện trong “Banned IP list”.

Với CrowdSec:

```
$sudo cscli decisions list
```

- Kết quả kỳ vọng: quyết định `type: ban` với lý do “ssh-bf” kèm thời gian hết hạn.
- Có thể đối chiếu bằng `sudo iptables -L -n` để thấy các ipset do Bouncer tạo ra.

Tổng kết chương

Những điểm cần ghi nhớ

- Bảo mật hiện đại dựa trên CIA và chuyển sang **Zero Trust**; nguy cơ lớn nhất thường đến từ **sai cấu hình** hơn là mã độc phức tạp.
- Tường lửa Linux (Netfilter/iptables) phải thiết kế theo nguyên tắc **Default Drop**: chặn mặc định, chỉ mở đúng cổng cần thiết.
- SSH hardening = vô hiệu hóa root + khóa Ed25519 thay mật khẩu + đổi cổng + whitelist IP – tuyến phòng thủ hiệu quả nhất chống brute-force.
- Fail2Ban lọc log theo ngưỡng thử sai; CrowdSec dùng threat intelligence cộng đồng để chặn cả botnet phân tán.